

Universidad Rafael Landívar.
Facultad de Ingeniería.
Licenciatura en Ingeniería en Informática y Sistemas.
Estructura de Datos I
Docente: Ing. Juan Pablo Valiente

MANUAL TÉCNICO

“Generador de Imágenes por Capas”

Cholotío Mendoza, Carlos Andrés
Carné: 1517925

1. Introducción

El presente documento detalla la arquitectura, el diseño algorítmico y la lógica interna de la aplicación "Generador de Imágenes por Capas". Este sistema fue desarrollado nativamente en C++ bajo el paradigma de Programación Orientada a Objetos (POO), cumpliendo con el requerimiento de implementar todas las estructuras de datos desde cero, garantizando un control absoluto sobre la memoria dinámica.

Todo el entorno de desarrollo, escritura de código y pruebas se trabajó íntegramente utilizando el editor **Visual Studio Code (VS Code)**. La responsabilidad de la asignación y liberación de memoria recae completamente en la creación de nodos propios, el uso estructurado de punteros, y la correcta aplicación de la instrucción delete para garantizar el rendimiento y evitar fugas de memoria (*memory leaks*), logrando así un sistema eficiente y totalmente autocontenido.

2. Arquitectura del Sistema y Gestión de Memoria

El sistema resuelve el problema del renderizado gráfico optimizado mediante un ecosistema de estructuras interconectadas. La arquitectura se diseñó bajo el principio de "No Redundancia Espacial":

- Las **Estructuras Primarias** (Matriz Dispersa, Lista Circular Global y Árboles) son las únicas dueñas de los objetos en memoria.
- Las **Estructuras Secundarias** (Listas simples dentro de Imágenes y Usuarios) no duplican información; únicamente almacenan referencias (punteros en C++) hacia los nodos de las estructuras primarias. Esto reduce drásticamente el consumo de RAM.

3. Estructuras de Datos Implementadas (Complejidad y Diseño)

3.1. Matriz Dispersa Ortogonal (Manejo de Capas) Es la estructura más compleja del sistema, encargada de almacenar los píxeles (códigos hexadecimales).

- **Diseño:** Implementada con cabeceras de filas (Row Headers) y columnas (Column Headers) dinámicas. Cada nodo interno posee cuatro punteros (up, down, left, right) para mantener la ortogonalidad.
- **Eficiencia Espacial:** Al no existir dimensiones fijas para los lienzos, el programa crea nodos exclusivamente en las coordenadas (X, Y) donde existe color. Los píxeles transparentes no ocupan espacio en memoria, logrando una eficiencia muy superior a la de un Array 2D tradicional.
- **Complejidad de Inserción:** $O(n + m)$, donde n y m son la cantidad de cabeceras de filas y columnas a recorrer para encontrar la posición ortogonal correcta. [AQUÍ: Inserta captura del diagrama de la Matriz Dispersa generado por Graphviz]

3.2. Árboles Binarios de Búsqueda (ABB) Se implementaron dos árboles para garantizar tiempos de búsqueda logarítmicos $O(\log n)$ en el mejor y caso promedio, facilitando la extracción de datos en milisegundos.

- **ABB de Capas:** Utiliza el ID numérico (int) como clave de ordenamiento. Menores a la izquierda, mayores a la derecha.
- **ABB de Usuarios:** Utiliza el nombre de usuario (std::string) como clave, aprovechando la sobrecarga de operadores de C++ (<, >) para ordenar los nodos alfabéticamente.
[AQUÍ: Inserta captura del diagrama del Árbol de Usuarios o de Capas]

3.3. Lista Circular Doblemente Enlazada (Catálogo de Imágenes)

- **Diseño:** Almacena todas las imágenes del sistema ordenadas por ID de forma ascendente. El puntero siguiente del último nodo se enlaza a la cabeza, y el puntero anterior de la cabeza apunta al último nodo.
- **Ventaja:** Permite un recorrido bidireccional ininterrumpido.

4. Algoritmos Críticos y Lógica de Negocio

4.1. Algoritmos de Recorrido de Árboles (Generación Limitada) Para la generación de imágenes por profundidad, el sistema implementa tres métodos de recorrido recursivo sobre el ABB de Capas:

- **Preorden (Raíz, Izquierda, Derecha):** Extrae las capas jerárquicamente desde la raíz hacia abajo.
- **Inorden (Izquierda, Raíz, Derecha):** Extrae las capas ordenadas numéricamente de menor a mayor.
- **Postorden (Izquierda, Derecha, Raíz):** Extrae primero las hojas del árbol y culmina en la raíz.

El algoritmo utiliza un contador por referencia (int&) que detiene la recursividad al alcanzar el límite de capas solicitado por el usuario.

4.2. Renderizado Visual (Integración Graphviz y HTML) El proceso de transformación de datos abstractos a una imagen .png visible consta de tres fases:

1. **Lienzo Virtual:** Se crea un Array 2D temporal inicializado en color blanco ("white").
2. **Superposición (Z-Index):** El algoritmo itera sobre la lista de capas de la imagen. Por cada capa, recorre su matriz dispersa y sobrescribe el color en el lienzo virtual. Las capas insertadas al final sobrescriben a las primeras.
3. **Traducción a DOT:** Se genera un archivo .dot donde cada nodo tiene el atributo shape=none y su contenido es una estructura <TABLE> de HTML. Cada celda <TD> representa un píxel coloreado con el atributo BGCOLOR. Graphviz compila este archivo nativamente.

4.3. Algoritmo de Eliminación en ABB (Módulo CRUD) El mantenimiento de usuarios exigió programar el algoritmo estándar de eliminación de Nodos en un ABB, evaluando tres casos críticos durante el tiempo de ejecución:

- **Caso 1 (Nodo Hoja):** Se libera la memoria y se actualiza el puntero del padre a nullptr.

- **Caso 2 (Nodo con un hijo):** El nodo padre adopta al único hijo del nodo a eliminar antes de liberar la memoria.
- **Caso 3 (Nodo con dos hijos):** Se busca el nodo sucesor inorden (el valor mínimo del subárbol derecho), se intercambia la información del usuario y se elimina recursivamente el nodo sucesor, manteniendo intacta la propiedad del árbol.

5. Módulo de Carga Masiva y Manejo de Excepciones

El sistema lee archivos de texto plano (.cap, .im, .usr) utilizando `std::ifstream`. Para blindar el software contra archivos corruptos o mal formateados, el analizador léxico (*parser*) implementa bloques try-catch. Si la función `std::stoi()` falla al intentar convertir una cadena alfanumérica inválida a un ID numérico, el sistema captura la excepción `std::invalid_argument`, reporta el error en consola y continúa con la siguiente línea, evitando una caída abrupta (*crash*) del programa.

6. Entorno de Compilación y Ejecución

Para que otro desarrollador pueda modificar o recompilar el proyecto, debe asegurar el siguiente entorno:

- **Lenguaje:** C++ (Estándar C++11 o superior recomendado).
- **Entorno de Desarrollo:** Visual Studio Code.
- **Compilador:** g++ (vía MinGW en entornos Windows) o equivalente en sistemas UNIX.
- **Librerías Permitidas:** Únicamente librerías estándar para flujos de entrada/salida y manipulación de cadenas (`<iostream>`, `<string>`, `<fstream>`, `<sstream>`).
- **Dependencia Gráfica:** El entorno del sistema operativo debe contar con los binarios de **Graphviz** agregados en la variable PATH del sistema. El código ejecuta el comando `system("dot -Tpng...")`, por lo que el CLI de Graphviz debe ser reconocible globalmente.